

SETNODE: Spatial Equivariant Transformer Neural ODE for Physical Dynamics

Trillyon Earl

Saint Louis University

Research mentors: Dr. Qingguang Guan and Dr. Zhifu Xie

The University of Southern Mississippi

Unpublished NSF REU technical report | Summer 2026

Revised October 3, 2026

Code repository: <https://github.com/Branshi/REU-PhysicalAI>

Abstract

We investigate SETNODE, a Spatial Equivariant Transformer Neural ODE for learning particle dynamics. Our graph processor combines an unnormalized sum of interaction messages with locally centered residual attention. An invariant scalar potential yields equivariant conservative forces, and velocity Verlet integration generates trajectories. We evaluate a force-supervised SETNODE model and five similarly sized baselines using 3,000 two-dimensional three-body trajectories and a reported four-hour training budget per model. The evaluated SETNODE model has 114,514 parameters, one interaction block, two attention heads, and latent width 64. On the same 50 held-out trajectories and 299 rollout transitions, its mean position MSE is 8.92×10^{-5} , 33.5% below our EGNN baseline, with no failures under the specified error threshold. Its mean final center-of-mass drift is 6.61×10^{-7} , close to the numerical reference, although true gravitational energy drift remains substantially larger. The findings concern the complete model-and-solver pipelines; they do not isolate attention’s contribution or establish robustness across training seeds.

1. Introduction

Learning physical dynamics requires a model to represent interactions among objects and remain accurate when its predictions are repeatedly integrated over time. Graph networks represent particles as nodes and interactions as edges (Battaglia et al., 2018; Sanchez-Gonzalez et al., 2020). Equivariant graph networks constrain predictions to transform consistently with the coordinate system (Satorras et al., 2021). Neural ODEs represent continuous-time dynamics with a learned vector field and use a numerical solver to generate trajectories (Chen et al., 2018). Energy-based models add structure by obtaining dynamics from derivatives of a learned scalar function (Greydanus et al., 2019).

We combine these ideas with attention (Vaswani et al., 2017; Veličković et al., 2018) in SETNODE (Figure 1). Our design has two complementary goals: to accumulate interaction information in a way motivated by physics, and to adjust that representation to the current configuration. The first goal follows from force additivity. Net force is a sum of

contributions, not an average whose weights must sum to one. We therefore keep an explicit unnormalized sum of learned interaction messages. This does not mean normalized attention cannot learn forces; our hypothesis is that an additive pathway makes their structure more natural to represent.

For the second goal, we apply the subtract-uniform centering operation studied by Ali et al. (2023) to local interaction messages. Each head subtracts the uniform neighbor weight from its softmax weights, so its output is an attention-weighted average minus a uniform average. This lets the model represent departures from treating all neighbors equally, while keeping a separate additive pathway. We intend attention to provide context-sensitive adjustments when particular interactions matter for predicting the dynamics, including uncommon configurations. Centering itself does not measure rarity, and the two branches are not forced to specialize in ordinary and unusual interactions.

A fluid confined in a box motivates future experiments. Suppose fluid–fluid interactions dominate the training samples, while fluid–wall interactions are less common. With suitable boundary features, attention could help represent these less prevalent but consequential interactions while the additive branch continues to aggregate all messages. This is a hypothesis for future fluid–boundary experiments, not a capability established by our three-body study. Such an extension also requires suitable modeling of boundary conditions and nonconservative dynamics.

Both branches act on latent features rather than independently identified physical forces. Their outputs contribute to an invariant potential, whose negative coordinate gradient supplies the learned force field. We use the three-body problem as a controlled setting for studying these representation and aggregation choices, not as a claim that a learned model should replace a numerical solver when the exact force law is already known.

Related work and scope. Ali et al. (2023) introduced subtract-uniform attention to address oversmoothing, in which repeated aggregation makes representations increasingly similar. Our centered heads use that same centering operation locally. Differential Transformer (Ye et al., 2025) also forms a

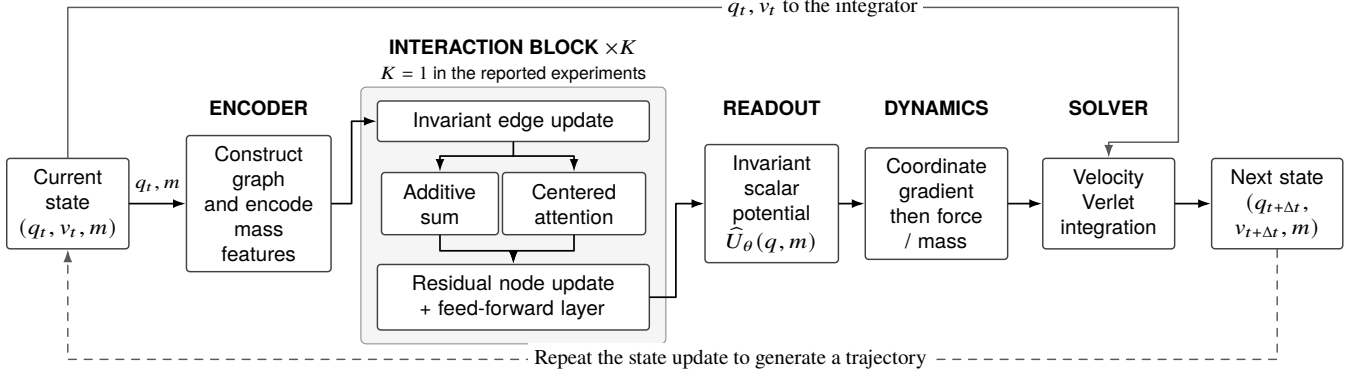


Figure 1: SETNODE architecture. We encode the particle graph, update its latent features through unnormalized-sum and centered-attention branches, and read out an invariant scalar potential. Differentiating the potential gives forces, which are divided by particle masses to obtain accelerations. Velocity Verlet uses this learned acceleration field and the current state to advance the system, reevaluating the force within each step. The processor supports K blocks; our evaluated model uses $K = 1$. Velocities enter the integrator, not the potential network. Repeating the state update generates a trajectory.

difference of attention maps, but subtracts a second learned softmax map rather than a uniform baseline.

Equivariant attention and continuous-time dynamics also have direct precedents. TorchMD-NET (Thölke and De Fabritiis, 2022) uses an equivariant transformer for molecular potentials and replaces softmax with SiLU in its attention weights. Thus attention need not use normalized weights. SEGNO (Liu et al., 2024) combines equivariant graph networks with second-order continuous-time dynamics. Our focus is the implemented combination of an explicit additive branch, locally centered attention, and an invariant-potential readout, together with a documented preliminary evaluation of rollout accuracy and conservation. Controlled ablations and repeated training are future tasks needed to identify which components provide reliable benefits.

2. Notation and Framework

We consider N particles with positions $q_i \in \mathbb{R}^d$, velocities $v_i \in \mathbb{R}^d$, and constant positive masses m_i . Our dataset uses $N = 3$, $d = 2$, and gravitational constant $G = 1$, without physical softening. Away from collisions, the dynamics and potential are

$$\begin{aligned} \dot{q}_i &= v_i, \\ \dot{v}_i &= G \sum_{j \neq i} m_j \frac{q_j - q_i}{\|q_j - q_i\|^3}, \\ U(q, m) &= -G \sum_{i < j} \frac{m_i m_j}{\|q_j - q_i\|}. \end{aligned} \quad (2.1)$$

The physical force is $F_i = -\nabla_{q_i} U = m_i \dot{v}_i$.

Our graph contains every directed edge $j \rightarrow i$ with $j \neq i$. Thus $\mathcal{N}(i) = \{j : j \neq i\}$ and $|\mathcal{E}| = N(N-1)$. We use raw masses as node features, $x_i = [m_i]$, and sender–receiver masses as edge features, $a_{ij} = [m_j, m_i]$. We standardize these features using means and standard deviations computed only from the training set. Learned MLP encoders then map them to 64-dimensional node and edge representations.

Geometry enters through pairwise radial quantities,

$$d_{ij}^2 = \|q_j - q_i\|^2, \quad r_{ij}^{-1} = \frac{1}{\|q_j - q_i\|}, \quad (2.2)$$

rather than raw coordinates passed to a generic MLP. In the implementation, the squared distance is clamped from below at 10^{-12} when computing the inverse distance. This numerical safeguard avoids division by zero and sufficiently small distances.

We write $Q \in O(d)$ for an orthogonal matrix, satisfying $Q^T Q = I$, and $b \in \mathbb{R}^d$ for a translation. A Euclidean coordinate change applies $q_i \mapsto Q q_i + b$ to every particle. The scalar potential should remain unchanged, while the forces should rotate or reflect with Q . Similarly, a particle permutation must leave the potential unchanged and permute the associated forces. We learn a position- and mass-dependent potential. The kinetic-energy term is known, and velocities enter the integrator rather than the potential network.

3. SETNODE Method

3.1. Invariant edge updates and dual aggregation

Let h_i and e_{ij} be the latent node and edge vectors, and let $\bar{h}_i = \text{LN}(h_i)$, where LN denotes layer normalization. A shared MLP updates each edge:

$$\tilde{e}_{ij} = e_{ij} + \phi_e(\bar{h}_i \parallel \bar{h}_j \parallel r_{ij}^{-1} \parallel d_{ij}^2 \parallel e_{ij}). \quad (3.1)$$

Here $\parallel$ denotes concatenation. The additive branch then computes

$$\begin{aligned} M_{i,\Sigma} &= \sum_{j \in \mathcal{N}(i)} \tilde{e}_{ij}, \\ \Delta_{i,\Sigma} &= \phi_\Sigma(\bar{h}_i \parallel M_{i,\Sigma}). \end{aligned} \quad (3.2)$$

This preserves an unnormalized sum of interaction messages. The nonlinear MLP does not require the branch’s output to be a sum of independent force contributions.

For attention head $a \in \{1, \dots, H\}$, we compute

$$\begin{aligned} z_{ij}^a &= W^a [\tilde{h}_i \parallel \tilde{h}_j \parallel \tilde{e}_{ij}] + b^a, \\ s_{ij}^a &= (w^a)^\top \text{LeakyReLU}_{0.2}(z_{ij}^a), \\ \alpha_{ij}^a &= \text{softmax}_{j \in \mathcal{N}(i)}(s_{ij}^a). \end{aligned} \quad (3.3)$$

The centered head output is

$$C_i^a = \sum_{j \in \mathcal{N}(i)} \left(\alpha_{ij}^a - \frac{1}{n_i} \right) V^a(\tilde{e}_{ij}), \quad n_i = |\mathcal{N}(i)|. \quad (3.4)$$

Here $V^a(e) = W_V^a e + b_V^a$ is a learned value projection. We concatenate the heads, apply a learned output projection, and compute the attention-branch update:

$$\begin{aligned} C_i &= W_O [C_i^1 \parallel \dots \parallel C_i^H] + b_O, \\ \Delta_{i,A} &= \phi_A(\tilde{h}_i \parallel C_i). \end{aligned} \quad (3.5)$$

The two branches are combined in a residual update,

$$\begin{aligned} u_i &= h_i + \Delta_{i,\Sigma} + \gamma \odot \Delta_{i,A}, \\ h_i^+ &= u_i + \text{FFN}(\text{LN}(u_i)), \end{aligned} \quad (3.6)$$

where $\odot$ is elementwise multiplication and FFN is a feed-forward network. We initialize every entry of the LayerScale vector γ to 10^{-2} (Touvron et al., 2021). The MLPs and feed-forward layer use SiLU activations; the attention scores use LeakyReLU.

The processor can contain K interaction blocks. Equations (3.1)–(3.6) describe one block; successive blocks update the features passed to the readout. Our evaluated model uses $K = 1$, latent and hidden widths of 64, two attention heads of width 32, and feed-forward width 128. Its configurable MLPs have one hidden layer. Table 2 and Appendix B summarize the saved settings.

3.2. Potential readout and physical forces

The potential readout has a separate residual edge update. Let h_i^{final} and e_{ij}^{final} denote the features after the last interaction block; for $K = 1$, these are h_i^+ and $\tilde{e}_{ij}$. With $\tilde{h}_i = \text{LN}(h_i^{\text{final}})$, we write

$$e_{ij}^R = e_{ij}^{\text{final}} + \phi_R(\tilde{h}_i \parallel \tilde{h}_j \parallel r_{ij}^{-1} \parallel d_{ij}^2 \parallel e_{ij}^{\text{final}}). \quad (3.7)$$

The scalar network output is

$$\begin{aligned} V_\theta(q, m) &= \sum_i \psi_{\text{node}}(\tilde{h}_i) \\ &\quad + \sum_{i < j} \psi_{\text{pair}}\left(\frac{e_{ij}^R + e_{ji}^R}{2} \parallel \rho(d_{ij}^2)\right). \end{aligned} \quad (3.8)$$

The radial embedding ρ has MLP widths $1 \rightarrow 32 \rightarrow 8$. Averaging the two directed messages makes each unordered pair contribution symmetric. Although the final readout is summed over particle pairs, each pair’s latent representation can contain information from other particles through message

passing. The learned potential can therefore represent many-body interactions rather than only independent pairwise terms.

The network predicts normalized force. Using the saved scalar scale $\sigma_F \simeq 1.412318$, we restore physical units:

$$\begin{aligned} \hat{F}_{i,\text{norm}} &= -\nabla_{q_i} V_\theta, & \hat{U}_\theta &= \sigma_F V_\theta, \\ \hat{F}_i &= \sigma_F \hat{F}_{i,\text{norm}} = -\nabla_{q_i} \hat{U}_\theta, & \hat{a}_i &= \hat{F}_i / m_i. \end{aligned} \quad (3.9)$$

A fixed scalar rescaling preserves the force’s geometric transformation properties. During training, we retain the computation graph of the coordinate derivative so that backpropagation can differentiate the force loss with respect to the network parameters.

3.3. Generating trajectories

Starting from (q_t, v_t) , we use velocity Verlet to advance the state:

$$\begin{aligned} a_t &= \hat{a}_\theta(q_t, m), \\ q_{t+\Delta t} &= q_t + \Delta t v_t + \frac{1}{2} \Delta t^2 a_t, \\ a_{t+\Delta t} &= \hat{a}_\theta(q_{t+\Delta t}, m), \\ v_{t+\Delta t} &= v_t + \frac{1}{2} \Delta t (a_t + a_{t+\Delta t}). \end{aligned} \quad (3.10)$$

We do not use temporal attention. For the simulated system, the current positions, velocities, and masses provide the state needed to determine its dynamics. Our network models spatial interactions at the current configuration, and repeated numerical integration generates the trajectory.

4. Structural Properties

The following propositions verify the intended properties of our construction, rather than introduce new general results about invariant potentials. We ignore floating-point rounding errors. Derivative-based statements apply where the learned potential is differentiable. Continuous-time results assume fixed model parameters and masses, along trajectories where the required derivatives exist. We give the proofs in Appendix A.

Proposition 4.1 (Centered attention head). *For a receiver i with $n_i > 0$ neighbors and head a ,*

$$\sum_{j \in \mathcal{N}(i)} \left(\alpha_{ij}^a - \frac{1}{n_i} \right) = 0. \quad (4.1)$$

Consequently,

$$C_i^a = \sum_j \alpha_{ij}^a V^a(\tilde{e}_{ij}) - \frac{1}{n_i} \sum_j V^a(\tilde{e}_{ij}). \quad (4.2)$$

The head output is zero when all neighbor value vectors are identical.

This property belongs to the centered head, not necessarily the full attention branch. The output projection has a bias, and ϕ_A also receives $\tilde{h}_i$. Thus $\Delta_{i,A}$ can be nonzero even when every head is zero. By contrast, the value-projection bias b_V^a cancels inside each head because its coefficients sum to zero.

This cancellation does not remove the output-projection bias b_O .

Proposition 4.2 (Invariant potential). *For any orthogonal matrix $Q \in O(d)$ and translation $b \in \mathbb{R}^d$,*

$$\widehat{U}_\theta(Qq + b, m) = \widehat{U}_\theta(q, m). \quad (4.3)$$

For a particle permutation P applied jointly to positions and masses,

$$\widehat{U}_\theta(Pq, Pm) = \widehat{U}_\theta(q, m). \quad (4.4)$$

The potential is unchanged by translating, rotating, reflecting, or consistently relabeling the system. Forces, however, must transform with the particles.

Proposition 4.3 (Equivariant force). *The force $\widehat{F}_i = -\nabla_{q_i} \widehat{U}_\theta$ satisfies*

$$\begin{aligned} \widehat{F}_i(Qq + b, m) &= Q\widehat{F}_i(q, m), \\ \widehat{F}_\theta(Pq, Pm) &= P\widehat{F}_\theta(q, m). \end{aligned} \quad (4.5)$$

Thus rotating or reflecting the system rotates or reflects its predicted forces, while relabeling particles relabels the outputs. The potential is permutation-invariant; the forces are permutation-equivariant.

Proposition 4.4 (Momentum and center of mass). *Translation invariance of $\widehat{U}_\theta$ implies*

$$\sum_i \widehat{F}_i = 0. \quad (4.6)$$

Under $\dot{q}_i = v_i$ and $m_i \dot{v}_i = \widehat{F}_i$, total momentum is constant and the center of mass moves at constant velocity:

$$\frac{d}{dt} \sum_i m_i v_i = 0, \quad \ddot{q}_{\text{cm}} = 0, \quad q_{\text{cm}} = \frac{\sum_i m_i q_i}{\sum_i m_i}. \quad (4.7)$$

The center of mass remains fixed only when its initial velocity is zero.

Proposition 4.5 (Conservation of learned energy). *Define the learned energy by*

$$\widehat{H}(q, v, m) = \sum_i \frac{1}{2} m_i \|v_i\|^2 + \widehat{U}_\theta(q, m). \quad (4.8)$$

Along the continuous-time dynamics $\dot{q}_i = v_i$ and $m_i \dot{v}_i = -\nabla_{q_i} \widehat{U}_\theta$, we have $d\widehat{H}/dt = 0$.

These properties do not guarantee an accurate physical force law. Conservation of the *learned* energy is distinct from conservation of the true gravitational energy. A finite-step numerical solver need not conserve either energy exactly.

5. Experimental Methods

5.1. Dataset and partitioning

Our dataset contains 3,000 two-dimensional three-body trajectories, each with 300 saved states at intervals of $\Delta t = 0.01$.

A seeded trajectory-level split assigns 2,100 trajectories to training, 450 to validation, and 450 to testing. The partitions are disjoint and cover the full dataset; all six checkpoints store the same split manifest. The accompanying generator uses velocity Verlet and shifts initial positions and velocities to the center-of-mass frame. Table 1 lists the saved generation settings.

Table 1: Saved three-body dataset settings, in simulation units. The generator applies the acceleration threshold to individual coordinate components.

Setting	Value
Gravitational constant G	1.0
Physical softening ε	0.0
Position / velocity initialization scales	0.8 / 0.8
Initial / trajectory separation thresholds	0.4 / 0.3
Maximum acceleration threshold	100.0
Rejected candidate trajectories	14,276
Generation / split seeds	42 / 42

5.2. Our baseline implementations

We implemented GNS, EGNN, EGNN-HNN, HNN, and a global MLP in our project codebase. They follow the general ideas of their model families, rather than reproducing the exact published benchmark configurations. The code and evaluation records name our EGNN baseline EGNS; we use *EGNN* throughout the paper.

GNS predicts acceleration from node features containing velocity and mass. Its eight edge features include relative displacement, radial quantities, sender mass, and analytic pairwise gravitational-acceleration components. **EGNN** uses mass and invariant radial features with geometric vector updates. **EGNN-HNN** combines graph processing with an invariant potential and coordinate differentiation.

Our **HNN** is a dense potential model. It takes six position coordinates and three masses, uses three width-235 Tanh hidden layers, and outputs a scalar. It does not impose Euclidean or permutation symmetry. Our **MLP** takes positions, velocities, and masses (15 inputs), uses three width-234 Tanh hidden layers, and predicts six acceleration coordinates.

As Table 2 shows, the six models have similar parameter counts but different inputs, depths, batch sizes, objectives, and integrators. We therefore evaluate complete model-and-solver pipelines under the same reported time budget. In particular, GNS receives analytic pairwise gravitational-acceleration features. It is a physics-feature-augmented baseline. These features are computed from the current state, rather than taken from future states.

5.3. Instantaneous supervision and model selection

We train the models to predict the instantaneous force or acceleration at sampled states. The checkpoint filenames call this stage `one_step`, but its loss fits neither the next state directly nor an entire trajectory. No rollout fine-tuning is included in the reported results.

Table 2: Evaluated model settings. For graph models, width is the node/edge latent dimension and the common hidden width of the configurable MLPs. For dense models, it is the number of units in each hidden layer. Blocks count interaction processors. Batch size is the number of states used per optimizer update. F and A denote force and acceleration targets. Parameter counts exclude normalization buffers.

Model	Width	Blocks	Batch size	Parameters	Target	Integrator
GNS	43	6	4	114,597	A	RK4
EGNN	40	6	4	116,006	A	RK4
EGNN-HNN	41	6	4	116,647	F	Velocity Verlet
HNN	235	—	256	113,506	F	Velocity Verlet
MLP	234	—	256	115,134	A	RK4
SETNODE	64	1	4	114,514	F	Velocity Verlet

Each model has a reported four-hour training budget on a MacBook M3 Max.¹ We use Adam with learning rate 10^{-4} and saved training seed 42. Each graph-model batch averages the loss over four sampled trajectory–time states, with 1,000 updates configured per epoch. The dense HNN and MLP trainers use batches of 256 states. We compute normalization statistics only on the training partition and save them with the models.

SETNODE, EGNN-HNN, and HNN use normalized force MSE:

$$\mathcal{L}_F = \text{MSE}\left(\widehat{F}_{\text{norm}}, \frac{ma}{\sigma_F}\right), \quad \sigma_F \simeq 1.412318. \quad (5.1)$$

Here ma is the physical force target and σ_F is its training-set normalization scale. GNS, EGNN, and MLP instead use normalized acceleration MSE:

$$\mathcal{L}_a = \text{MSE}\left(\widehat{a}_{\text{norm}}, \frac{a}{\sigma_a}\right), \quad \sigma_a \simeq 1.130909, \quad (5.2)$$

with zero acceleration-target mean. We also record acceleration errors in physical units for all models. Force and acceleration supervision weight the examples differently when masses vary. Since $\widehat{F}_i - F_i = m_i(\widehat{a}_i - a_i)$, each particle’s contribution to normalized force MSE is proportional to

$$\frac{m_i^2}{\sigma_F^2} \|\widehat{a}_i - a_i\|^2. \quad (5.3)$$

Force supervision therefore emphasizes acceleration errors for higher-mass particles. Our SETNODE checkpoint uses the force objective, not the hybrid force–acceleration objective in separate experimental variants.

All six models use the same ordered set of 500 validation trajectory–time pairs, selected with validation seed 1234. We keep each model’s checkpoint with the lowest recorded normalized validation objective for its target. The evaluation uses these saved one-step checkpoints.

5.4. Rollout protocol and metrics

We use the first 50 trajectories in the saved test-manifest order. Every model starts from the same true initial position and

velocity for each trajectory and predicts $T = 299$ transitions, covering 2.99 simulation-time units. We exclude the initial state from prediction error. For trajectory r ,

$$E_r^q = \frac{1}{TNd} \sum_{t=1}^T \sum_{i=1}^N \|\widehat{q}_{r,t,i} - q_{r,t,i}\|^2. \quad (5.4)$$

We report the mean, population standard deviation, and median of E_r^q across the 50 trajectories. We count a trajectory as a failure if a predicted position, velocity, or acceleration becomes NaN or infinite, or if the maximum position RMSE over the rollout exceeds 0.1. We measure acceleration error on all 300 saved true states, separately from the rollout error.

Conservation diagnostics use the true gravitational energy,

$$H(q, v, m) = \sum_i \frac{1}{2} m_i \|v_i\|^2 + U(q, m), \quad (5.5)$$

not the learned potential. We record final absolute energy drift $|H_T - H_0|$, relative energy drift $|H_T - H_0|/\max\{|H_0|, 10^{-8}\}$, and center-of-mass displacement $\|q_{\text{cm},T} - q_{\text{cm},0}\|$. The latter is interpreted for the generator’s zero-momentum initialization. Applying the same diagnostics to the saved numerical trajectories gives a reference. We compute the reported summaries from the saved per-trajectory evaluation records.

6. Empirical Results

6.1. Prediction accuracy and failures

SETNODE has the lowest mean and median rollout position MSE in our fixed test suite (Table 3). Its mean error is 33.5% below EGNN and 61.8% below EGNN-HNN. The advantage is not uniform: SETNODE has lower error than EGNN on 30 of 50 paired trajectories and lower error than EGNN-HNN on 32 of 50.

The error distributions overlap substantially among the equivariant models, and some trajectories have much larger errors than the median (Figure 2). SETNODE and EGNN have no failures under our threshold. None of the six models has a recorded NaN or infinite prediction in this suite: all recorded failures exceed the position-error threshold. Zero failures on these 50 trajectories does not guarantee stability on other trajectories or longer rollouts.

¹The archived checkpoints do not record exact elapsed training time or completed epoch counts, so the four-hour duration is taken from the experiment record rather than independently verified.

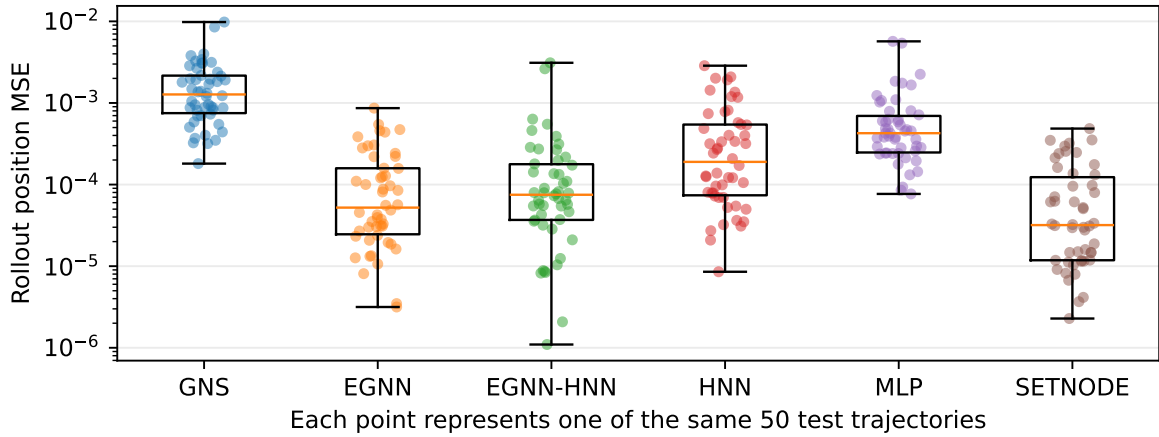


Figure 2: Distribution of rollout position errors. Each dot represents one of the same 50 test trajectories; horizontal jitter is used only for visibility. Boxes show the interquartile range and median, and whiskers span the observed range. The vertical axis is logarithmic. We show one evaluated checkpoint per architecture.

Table 3: Rollout position MSE on the same 50 trajectories. SD is the population standard deviation across trajectories. Failures follow Section 5.4.

Model	Mean	SD	Median	Failures
GNS	1.81×10^{-3}	1.81×10^{-3}	1.27×10^{-3}	14/50
EGNN	1.34×10^{-4}	1.74×10^{-4}	5.22×10^{-5}	0/50
EGNN-HNN	2.33×10^{-4}	5.57×10^{-4}	7.50×10^{-5}	1/50
HNN	4.71×10^{-4}	6.34×10^{-4}	1.90×10^{-4}	4/50
MLP	7.61×10^{-4}	1.09×10^{-3}	4.26×10^{-4}	4/50
SETNODE	8.92×10^{-5}	1.14×10^{-4}	3.19×10^{-5}	0/50

Table 4: Mean final conservation diagnostics. Absolute energy drift and center-of-mass drift use simulation units; relative drift is dimensionless. The numerical reference uses the saved true-trajectory diagnostics from SETNODE’s evaluation.

Model	Absolute energy drift	Relative energy drift	Center of mass drift
GNS	5.07×10^{-2}	5.08×10^{-1}	7.11×10^{-2}
EGNN	7.71×10^{-3}	6.36×10^{-2}	7.82×10^{-3}
EGNN-HNN	1.32×10^{-2}	1.85×10^{-1}	6.04×10^{-7}
HNN	1.49×10^{-2}	1.33×10^{-1}	5.24×10^{-3}
MLP	4.03×10^{-2}	5.90×10^{-1}	2.04×10^{-2}
SETNODE	7.35×10^{-3}	1.51×10^{-1}	6.61×10^{-7}
Numerical reference	7.58×10^{-5}	1.01×10^{-3}	6.19×10^{-7}

6.2. Conservation behavior

SETNODE and EGNN-HNN have mean final center-of-mass drifts of approximately 6×10^{-7} , close to the numerical reference (Table 4). This is consistent with the zero total force implied by a translation-invariant potential. It does not establish accurate relative motion: an incorrect force field can still preserve the center of mass.

SETNODE’s mean absolute energy drift is 7.35×10^{-3} , slightly below EGNN’s but about 97 times the numerical reference. The ordering changes for mean *relative* energy drift: EGNN obtains 0.0636 and SETNODE obtains 0.151. SETNODE’s median relative drift is 0.00487, much smaller

than its mean; its largest value is 5.94 on trajectory 2085. Relative normalization can emphasize trajectories with small initial energy, so absolute and relative drift answer different questions.

Imperfect force fitting and finite-step integration can both contribute to energy drift. Our experiment does not separate their effects. In particular, conservation of the learned energy in continuous time does not imply conservation of the true gravitational energy measured here.

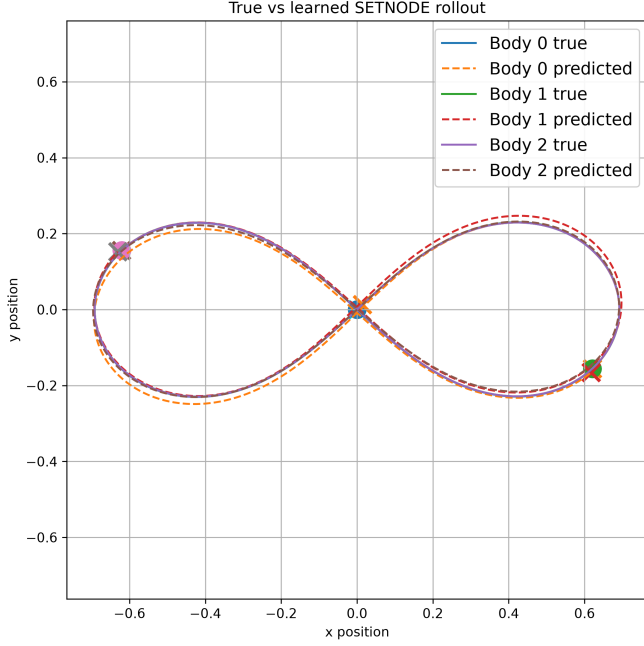
6.3. Illustrative rollouts with an extended potential readout

Figure 3 shows two illustrative trajectories from later SETNODE variants trained for approximately 12 hours on a M3 Macbook Max. These variants use an extended potential readout with four changes. First, we apply the shared pair-scoring MLP to the two directed pair representations separately, then average the resulting scalar scores, rather than averaging the representations before scoring. Second, we use a multiscale distance embedding. Third, skip connections give the readout direct access to the original normalized node and edge features alongside the processed latents. Finally, learned gates combine node, pair, and global energy contributions, with the global term computed from pooled latent features.

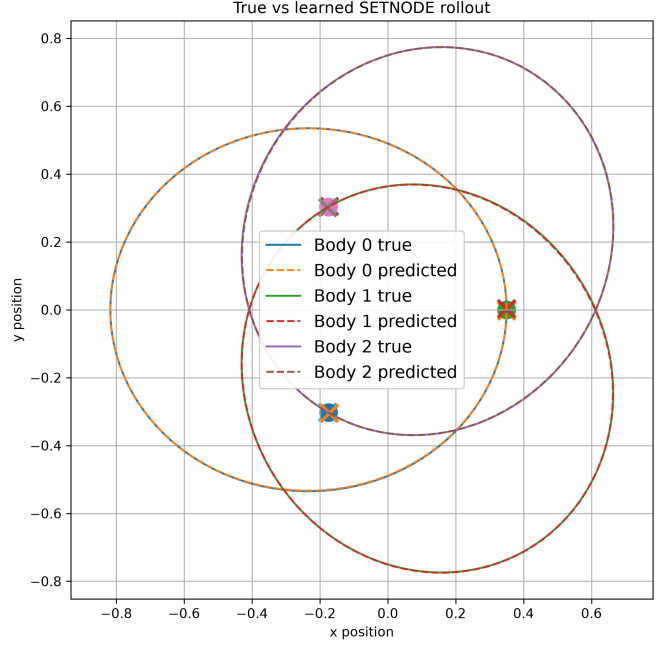
These examples are separate from the four-hour evaluation and are not included in its summary statistics. Because both the training duration and readout differ, these images do not isolate the effects of either change.

7. Discussion and Limitations

Our results support further study of the complete SETNODE design, but do not establish that centered attention causes the improvement. EGNN-HNN has different depth, width, and other implementation details; it is not SETNODE with attention removed. The models also differ in supervised targets, inputs, and solvers. We cannot attribute the observed error differences to one component alone.



(a) Figure-eight trajectory, $t = 3.24$.



(b) Three-body orbital trajectory, $t = 3.68$.

Figure 3: Illustrative rollouts with an extended SETNODE readout. The models were trained for approximately 12 hours on a M3 Macbook Max and use scalar-score symmetrization, multiscale distance features, input-feature skip connections, and gated node, pair, and global contributions. Reference and predicted trajectories are overlaid; circles and crosses mark their respective positions at the displayed final time. These qualitative examples are separate from the four-hour quantitative evaluation.

The evidence covers one saved checkpoint per architecture and one fixed set of 50 test trajectories. Trajectory-level standard deviations describe variation within that set. The suite uses only part of the 450-trajectory test partition. We have not established a robust architecture ranking or generalization to different particle counts, mass distributions, or much longer horizons. The rejection thresholds also limit coverage of close encounters.

Our additive/context-sensitive division remains a design hypothesis. Both branches operate on latent features, and neither is forced to learn a particular class of physical interactions. With exactly two neighbors per particle, the present task does not test variable-neighborhood behavior.

Our priority is an exact ablation using the same SETNODE implementation: additive-only processing, additive processing with ordinary attention, and additive processing with centered attention. Removing the additive branch would test its role separately. These experiments should keep the force objective, solver, split, and validation procedure fixed, use several training seeds, and report differences in active parameters and computational cost. A geometric-only GNS variant would help distinguish feature engineering from message-passing design. None of these ablation results is reported here.

Implementation checks should also measure rotation and permutation equivariance, summed internal force, and agreement between automatic differentiation and finite differences of the potential. Our propositions establish mathematical properties but do not replace numerical tests of the software. Rollout-fine-tuning code and separate checkpoints exist, but

matching evaluation results are outside this report.

For the fluid-boundary motivation in Section 1, future experiments should measure errors separately near walls and in the fluid interior while varying the prevalence of boundary interactions in training. Those studies require appropriate boundary features and nonconservative modeling. Other extensions include mass-ratio shifts, particle-count transfer, and three-dimensional trajectories.

8. Conclusion

We developed SETNODE to combine physics-motivated additive aggregation with context-sensitive centered attention and an invariant-potential force model. Our original width-64, one-block, two-head checkpoint achieves a mean rollout position MSE of 8.92×10^{-5} on the fixed 50-trajectory suite, with center-of-mass drift close to the numerical reference. Energy drift and overlapping error distributions qualify this result. Controlled ablations, repeated training, and broader rollout tests are needed to determine which components provide reliable benefits.

Acknowledgments

This work was conducted through The University of Southern Mississippi REU Site in Deep Learning for Dynamical Systems. I thank Dr. Qingguang Guan and Dr. Zhifu Xie for their mentorship and feedback. This material is based upon work supported by the National Science Foundation REU Site under Grant (FAIN) 2447675.

References

- Ameen Ali, Tomer Galanti, and Lior Wolf. Centered self-attention layers. *arXiv:2306.01610*, 2023. <https://arxiv.org/abs/2306.01610>.
- Peter W. Battaglia et al. Relational inductive biases, deep learning, and graph networks. *arXiv:1806.01261*, 2018.
- Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David K. Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems*, volume 31, 2018.
- Samuel Greydanus, Misko Dzamba, and Jason Yosinski. Hamiltonian neural networks. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- Yang Liu, Jiashun Cheng, Haihong Zhao, Tingyang Xu, Peilin Zhao, Fuguee Tsung, Jia Li, and Yu Rong. SEGNO: Generalizing equivariant graph neural networks with physical inductive biases. In *International Conference on Learning Representations*, 2024. <https://arxiv.org/abs/2308.13212>.
- Alvaro Sanchez-Gonzalez, Jonathan Godwin, Tobias Pfaff, Rex Ying, Jure Leskovec, and Peter Battaglia. Learning to simulate complex physics with graph networks. In *ICML*, volume 119 of *Proceedings of Machine Learning Research*, pages 8459–8468, 2020.
- Víctor García Satorras, Emiel Hooeboom, and Max Welling. E(n) equivariant graph neural networks. In *ICML*, volume 139 of *Proceedings of Machine Learning Research*, pages 9323–9332, 2021.
- Philipp Thölke and Gianni De Fabritiis. Equivariant transformers for neural network based molecular potentials. In *International Conference on Learning Representations*, 2022. <https://arxiv.org/abs/2202.02541>.
- Hugo Touvron, Matthieu Cord, Alexandre Sablayrolles, Gabriel Synnaeve, and Hervé Jégou. Going deeper with image transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 32–42, 2021.
- Ashish Vaswani et al. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations*, 2018.
- Tianzhu Ye, Li Dong, Yuqing Xia, Yutao Sun, Yi Zhu, Gao Huang, and Furu Wei. Differential transformer. In *International Conference on Learning Representations*, 2025. <https://arxiv.org/abs/2410.05258>.

A. Proofs of the Structural Properties

We prove the propositions from Section 4 in the same order. All sums over neighbors are taken over $j \in \mathcal{N}(i)$. The derivative arguments assume the potential is differentiable at the configurations involved. For the continuous-time results, model parameters and particle masses are fixed, and the trajectory remains where the required derivatives exist.

A.1. Proof of Proposition 4.1: centered attention

Proof. The softmax weights add to one. The n_i uniform weights each equal $1/n_i$, so they also add to one. Subtracting the two sums gives

$$\sum_j \left(\alpha_{ij}^a - \frac{1}{n_i} \right) = \sum_j \alpha_{ij}^a - \frac{n_i}{n_i} = 1 - 1 = 0.$$

Expanding the centered head in Equation (3.4) gives

$$C_i^a = \sum_j \alpha_{ij}^a V^a(\tilde{e}_{ij}) - \frac{1}{n_i} \sum_j V^a(\tilde{e}_{ij}).$$

The first term is the learned attention-weighted average and the second is the uniform average. Finally, if every neighbor has the same value vector $V^a(\tilde{e}_{ij}) = c$, then

$$C_i^a = c \sum_j \left(\alpha_{ij}^a - \frac{1}{n_i} \right) = 0.$$

Thus the head contributes no correction when all neighbor values are identical. $\square$

A.2. Proof of Proposition 4.2: invariant potential

Proof. We first consider a coordinate change $q'_i = Qq_i + b$. Translating both particles by the same vector cancels in their relative position. Since $Q^\top Q = I$,

$$\|q'_j - q'_i\|^2 = \|Q(q_j - q_i)\|^2 = (q_j - q_i)^\top Q^\top Q (q_j - q_i) = \|q_j - q_i\|^2.$$

Every radial quantity used by the network is therefore unchanged. This includes the inverse-distance feature and its numerical floor. The masses and their saved normalization statistics are also unchanged, so the encoded node and edge features are the same.

Now follow the features through the network. The shared edge MLP receives the same inputs and returns the same edge updates. The incoming sums, attention scores, centered heads, and node updates therefore remain unchanged as well. Repeating this argument for each processing layer shows that all latent features are unchanged. The final node and pair sums in Equation (3.8) give the same scalar V_θ . Multiplication by the fixed scalar σ_F also preserves this equality, so

$$\widehat{U}_\theta(Qq + b, m) = \widehat{U}_\theta(q, m).$$

Next, permute the particles and their masses consistently. A node or edge then receives a different label but carries the same physical data. Because the encoders and update functions share their parameters across particles and edges, the intermediate representations are permuted in the same way. A sum does not depend on the order of its terms, and neighbor-wise softmax permutes its weights when its inputs are permuted.

The final node sum includes the same terms in a different order. Each unordered-pair term is also unchanged: averaging the two directed messages removes the orientation of that pair. Hence the potential satisfies $\widehat{U}_\theta(Pq, Pm) = \widehat{U}_\theta(q, m)$. $\square$

A.3. Proof of Proposition 4.3: equivariant force

Proof. Let $q' = Qq + b$. Proposition 4.2 gives $\widehat{U}_\theta(Qq + b, m) = \widehat{U}_\theta(q, m)$. Differentiating with respect to the original particle position q_i , the chain rule gives

$$Q^\top \nabla_{q'_i} \widehat{U}_\theta(q', m) = \nabla_{q_i} \widehat{U}_\theta(q, m).$$

Multiply by Q and use $QQ^\top = I$:

$$\nabla_{q'_i} \widehat{U}_\theta(q', m) = Q \nabla_{q_i} \widehat{U}_\theta(q, m).$$

The force is the negative gradient of the potential. Negating the last equality therefore yields $\widehat{F}_i(Qq + b, m) = Q \widehat{F}_i(q, m)$.

For particle permutations, regard P as permuting the particle blocks of the stacked coordinate vector. Applying the chain rule to $\widehat{U}_\theta(Pq, Pm) = \widehat{U}_\theta(q, m)$, with masses held fixed with respect to coordinate differentiation, gives

$$P^\top \nabla_{q'} \widehat{U}_\theta(q', Pm) = \nabla_q \widehat{U}_\theta(q, m), \quad q' = Pq.$$

Since $PP^\top = I$, multiplying by P and negating gives $\widehat{F}_\theta(Pq, Pm) = P \widehat{F}_\theta(q, m)$. Relabeling the particles therefore relabels their forces. $\square$

A.4. Proof of Proposition 4.4: momentum and center of mass

Proof. Translate every particle by sc , where $c \in \mathbb{R}^d$ is arbitrary and s is a scalar translation parameter. Translation invariance means

$$\widehat{U}_\theta(q_1 + sc, \dots, q_N + sc, m) = \widehat{U}_\theta(q, m).$$

The right-hand side does not depend on s . Differentiating at $s = 0$ gives

$$0 = c^\top \sum_i \nabla_{q_i} \widehat{U}_\theta(q, m).$$

This holds for every vector c , which is possible only if the summed gradient is zero. Because $\widehat{F}_i = -\nabla_{q_i} \widehat{U}_\theta$, we obtain $\sum_i \widehat{F}_i = 0$.

Total momentum is $p = \sum_i m_i v_i$. Using the equations of motion and constant masses,

$$\frac{dp}{dt} = \sum_i m_i \dot{v}_i = \sum_i \widehat{F}_i = 0.$$

Thus momentum stays constant. The center of mass satisfies

$$\dot{q}_{\text{cm}} = \frac{\sum_i m_i v_i}{\sum_i m_i} = \frac{p}{\sum_i m_i}, \quad \ddot{q}_{\text{cm}} = 0.$$

It moves at constant velocity and remains fixed only when that velocity is initially zero. □

A.5. Proof of Proposition 4.5: learned energy

Proof. We differentiate the kinetic and learned potential terms separately:

$$\frac{d\widehat{H}}{dt} = \sum_i m_i v_i^\top \dot{v}_i + \sum_i (\nabla_{q_i} \widehat{U}_\theta)^\top \dot{q}_i.$$

Substitute $m_i \dot{v}_i = \widehat{F}_i$, $\dot{q}_i = v_i$, and $\nabla_{q_i} \widehat{U}_\theta = -\widehat{F}_i$:

$$\frac{d\widehat{H}}{dt} = \sum_i v_i^\top \widehat{F}_i - \sum_i \widehat{F}_i^\top v_i = 0.$$

The two sums are equal because each dot product is a scalar. Any change in kinetic energy is therefore canceled by the change in the learned potential energy. □

This is a statement about the learned continuous-time system. It neither identifies the learned potential with the true gravitational potential nor establishes exact energy conservation after finite-step numerical integration.

B. Experimental Record and Reproducibility

B.1. Archived checkpoints and evaluation records

We use the six files named `one_step_115k_4h.pt` in the `gns`, `egns`, `egnn_hnn`, `hnn`, `setnode`, and `mlp` checkpoint directories. We read the architecture settings, tensor shapes, target scales, validation criteria, and embedded split manifests from these checkpoints. They contain the same original three-body split and the same ordered validation-state list. The files `<model>_test_suite.json` supply the numerical results. Their model-relative checkpoint paths and trajectory IDs match the archived model names and test manifest, and their summaries agree with recomputation from the individual entries.

Table 5: Additional saved SETNODE settings and checkpoint identifiers. The source bundle contains full configurations and hashes.

SETNODE setting	Value	Model	Checkpoint SHA-256 prefix
Node / edge input dimensions	1 / 2	GNS	8c3785e9b1
Hidden layers per configurable MLP	1	EGNN	23236a497e
Attention heads / width per head	2 / 32	EGNN-HNN	44c23e8052
Feed-forward width	128	HNN	3d635ef670
Readout distance embedding	8	MLP	1192eb5d2f
Validation samples / seed	500 / 1234	SETNODE	b293fea95e

B.2. Source version and remaining gaps

Our method description follows the REU-PhysicalAI source snapshot `32c05fe6` and the supplied MLP files found in snapshot `67a9d90`. This is the inspected source version, not a verified historical training commit. The original checkpoint’s normalization and validation fields, together with the matching trainer, support the force objective.

The archive does not store exact training times, completed-update counts, the original execution environment and commands, mass-sampling parameters, and simulation/storage precision. The large saved epoch limits are configured limits, not completed epoch counts. We also do not infer the missing dataset settings from later code defaults.

The numerical analyses in this paper use archived evaluation records; we did not retrain the models or rerun the physical rollouts for this analysis.